

Project Proposal & Design Document

SwayaLang: A Domain-Specific Language and Compiler for **Backtestable Stock-Trading Strategies**

Name	Swayam Shukla
Register Number	24BCE0751
Course	Compiler Design Laboratory (BCSE307P)
Submission	Phase 1 — Problem Definition and System Design
Project Type	Individual Project
Institution	VIT Vellore

1. Abstract

SwayaLang is a small, statically-checked domain-specific language (DSL) for expressing rule-based stock-trading strategies, paired with a compiler/interpreter that executes those strategies against historical price data. Rather than writing ad-hoc scripts to test a trading idea, a user expresses entry, exit, and risk-management logic declaratively (e.g. technical-indicator crossovers, threshold conditions, and stop-loss rules), and the system lexes, parses, semantically validates, optimizes, and executes that specification, producing a simulated trade log and performance report. The project exercises the complete compiler pipeline — lexical analysis, syntax analysis, AST construction, semantic analysis with a scoped symbol table, intermediate representation, code optimization, and interpretation — anchored in a concrete, demonstrable, and independently verifiable domain.

2. Problem Statement

Retail traders and finance students who want to test a trading idea (e.g. "buy when the 50-day moving average crosses above the 200-day moving average and RSI is below 30") are usually forced to either hand-write imperative backtesting code for every new idea, or rely on opaque, closed-source strategy builders in commercial platforms. There is no lightweight, transparent way to express a strategy declaratively, have it validated for correctness before it runs, and have it compiled down to an efficient, reusable evaluation form. This project addresses that gap by treating a trading strategy as a program in its own right — one with a formal grammar, a type system, and a compilation pipeline — rather than as loosely-structured script code.

3. Motivation

- Trading-strategy logic is naturally rule-based and declarative, making it a strong real-world fit for DSL design — a more compelling and demonstrable subject than a generic arithmetic or toy language.
- It requires genuine semantic analysis (type-checking indicators, catching illogical rules such as a stop-loss above the entry price) rather than a syntax-only exercise.
- It naturally motivates a concrete optimization pass (common-subexpression elimination across rules that reuse the same indicator), giving the project a defensible, measurable "code optimization" story.
- Historical price data is freely available (e.g. Yahoo Finance CSV exports), so the project is fully testable end-to-end without any data-access blockers.

- The output (a backtest report with win rate, drawdown, and returns) gives an unambiguous, numeric way to demonstrate correctness during review.

4. Objectives

- Design a formal grammar for a rule-based trading-strategy DSL covering indicators, comparators, boolean conditions, and order actions (BUY / SELL / SHORT / STOP-LOSS).
- Implement a lexer and recursive-descent (or LL(1)) parser that builds an Abstract Syntax Tree from strategy source text.
- Implement semantic analysis with a symbol table that tracks declared indicators/variables per strategy and enforces type and logical-consistency rules.
- Translate validated strategies into an intermediate representation (expression trees / three-address code) suitable for repeated evaluation.
- Implement at least one meaningful optimization pass (constant folding and/or common-subexpression elimination on shared indicator calculations).
- Build an interpreter/execution engine that walks historical OHLCV (Open-High-Low-Close-Volume) data bar-by-bar, evaluates compiled rules, and simulates a trading portfolio.
- Generate a backtest report (trade log, win rate, max drawdown, cumulative return) as the final, demonstrable output.
- Implement meaningful lexical, syntax, and semantic error detection with informative messages.

5. Scope

In scope:

- A defined subset of technical indicators: Simple Moving Average (SMA), Exponential Moving Average (EMA), Relative Strength Index (RSI), and raw price/volume fields.
- Single-instrument strategies evaluated over historical daily OHLCV data supplied as CSV.
- Long and short positions, with stop-loss and take-profit conditions.
- A command-line / script-driven interpreter (no live brokerage integration).

Out of scope:

- Live trading, order execution, or brokerage API integration.
- Multi-asset portfolio optimization or options/derivatives strategies.
- Real-time streaming data ingestion.

6. Background Study

The design draws on standard compiler-construction theory (lexical analysis via finite automata, recursive-descent/LL(1) parsing, AST-based semantic analysis, and three-address-code intermediate representations) as covered in the Compiler Design course. On the domain side, it draws on well-known technical-analysis concepts (moving-average crossovers, RSI-based overbought/oversold signals, stop-loss risk management) that are standard, publicly documented trading heuristics — no proprietary or confidential trading logic is used. Existing commercial strategy-builder tools (e.g. TradingView's Pine Script) demonstrate that DSL-based strategy specification is a proven and valuable pattern, which this project reimplements from first principles for compiler-design learning purposes.

7. Compiler Design Concepts Involved

Compiler Phase	Application in SwayaLang
Lexical Analysis	Tokenizing keywords (BUY, SELL, IF, STOP-LOSS), indicator names, identifiers, numeric literals, comments, and operators from raw strategy source text.
Syntax Analysis	Grammar for strategy blocks, compound boolean conditions, and multi-leg rules (entry + exit + stop-loss).
AST Construction	Building an Abstract Syntax Tree representing each strategy's conditions and actions for downstream analysis and execution.
Symbol Table	Tracking declared indicators/variables per strategy, with scope handling for multi-strategy scripts.
Semantic Analysis	Type-checking (numeric vs. indicator vs. ticker), undefined-variable detection, and logical-consistency checks (e.g. stop-loss above entry price).
Intermediate Code	Lowering validated ASTs into expression trees / three-address code for repeated, efficient evaluation.
Code Optimization	Constant folding on literal thresholds; common-subexpression elimination when multiple rules reuse the same indicator.
Interpretation / Execution	A runtime that evaluates compiled strategies bar-by-bar against historical data and simulates trade execution.
Error Handling	Detection and reporting of lexical, syntax, and semantic errors with source-position information.

8. Proposed Methodology

The system will be developed as a classical multi-pass compiler/interpreter pipeline, implemented incrementally across the three project phases:

- Phase 1 (this document): Grammar design, language specification, and an initial prototype lexer/parser capable of tokenizing and parsing a minimal strategy.
- Phase 2: Full parser and AST construction, symbol table, semantic analysis, and a first working interpreter that can evaluate a single strategy against sample data.
- Phase 3: Intermediate representation, optimization passes, complete error handling, backtest reporting, and final integration and testing across edge cases (invalid syntax, undefined indicators, boundary conditions such as empty datasets).

8.1 Sample Language Snippet

```
STRATEGY "GoldenCross" {  
  INDICATOR sma_50 = SMA(CLOSE, 50)  
  INDICATOR sma_200 = SMA(CLOSE, 200)  
  INDICATOR rsi_14 = RSI(CLOSE, 14)  
  
  ENTRY: BUY 100 SHARES OF "AAPL" IF sma_50 > sma_200 AND rsi_14 < 30  
  EXIT: SELL ALL IF sma_50 < sma_200  
  STOP_LOSS: SELL ALL IF PRICE < ENTRY_PRICE * 0.95  
}
```

8.2 Illustrative Grammar (EBNF, abbreviated)

```
strategy      := 'STRATEGY' STRING '{' indicator_decl* rule_block+ '}'
indicator_decl := 'INDICATOR' IDENT '=' func_call
func_call     := IDENT '(' arg (',' arg)* ')'
rule_block    := ('ENTRY'|'EXIT'|'STOP_LOSS') ':' action 'IF' condition
action        := ('BUY'|'SELL'|'SHORT') (NUMBER 'SHARES' 'OF')? STRING | 'SELL' 'ALL'
condition     := condition ('AND'|'OR') condition | expr comparator expr | '(' condition ')'
comparator    := '>' | '<' | '>=' | '<=' | '=='
expr          := IDENT | NUMBER | expr ('*'|'/'|'+'|'-') expr
```

9. System Architecture

The system follows a standard staged compiler architecture, shown below as the data flow from raw strategy source through to a backtest report.

Stage	Input	Output
1. Lexer	Strategy source (.strat file)	Token stream
2. Parser	Token stream	Abstract Syntax Tree (AST)
3. Semantic Analyzer	AST	Annotated AST + populated Symbol Table
4. IR Generator	Annotated AST	Expression trees / three-address code
5. Optimizer	IR	Optimized IR (folded constants, shared sub-expressions)
6. Interpreter / Runtime	Optimized IR + historical OHLCV data (CSV)	Trade log
7. Report Generator	Trade log	Backtest report: win rate, drawdown, returns

Module boundaries are kept strict — each stage consumes only the output of the previous stage — so individual phases can be unit-tested in isolation (e.g. feeding hand-crafted token streams directly to the parser) as required by the lab's testing expectations.

10. Technology Stack

Component	Choice	Rationale
Implementation Language	Python 3	Fast iteration for lexer/parser logic; rich ecosystem for numeric/data tooling.
Lexer/Parser Approach	Hand-written recursive-descent parser	Gives full control and deeper understanding of parsing theory, as required by the lab.
Data Handling	pandas	Efficient loading and manipulation of historical OHLCV CSV data.
Testing	pytest	Unit testing of lexer, parser, semantic analyzer, and interpreter modules.
Version Control	Git + GitHub	Maintain development history and a project log, as recommended by the course.

Visualization (optional)	matplotlib	Plotting the equity curve from backtest results.
--------------------------	------------	--

11. Initial Prototype Plan

As required for the Phase 1 deliverable, an initial working prototype will be developed covering the earliest stage of the pipeline:

- A lexer capable of tokenizing a minimal single-rule strategy (keywords, one indicator call, one comparator, one numeric literal).
- A minimal recursive-descent parser that accepts the tokenized output and builds a partial AST for a single ENTRY rule.
- A basic driver script that reads a .strat file, prints the token stream, and prints the resulting AST in a readable nested form — demonstrating that the pipeline is functional end-to-end at a small scale, ready for expansion in Phase 2.

12. Expected Outcomes

- A fully functional DSL compiler/interpreter for rule-based trading strategies, demonstrable on real historical data.
- A backtest report generator producing win rate, maximum drawdown, and cumulative return for any compiled strategy.
- A documented grammar specification and architecture suitable for the final project report.
- Demonstrated error handling for malformed strategies (lexical, syntax, and semantic errors) with clear, source-located error messages.

13. Version Control & Development Log

The project will be maintained in a Git repository (GitHub) from Phase 1 onward, with incremental commits corresponding to each pipeline stage (lexer, parser, semantic analyzer, IR/optimizer, interpreter, reporting) and a running development log documenting design decisions, challenges, and fixes, as recommended by the course's code development guidelines.